Exercise1:

```
import java.util.*;
class Product {
   int productId;
   String productName;
   int quantity;
   double price;
   Product(int id, String name, int qty, double price) {
      productId = id;
      productName = name;
      quantity = qty;
      this.price = price;
   }
   public String toString() {
      return productId + " " + productName + " " + quantity + " " + price;
   }
}
public class InventoryManagement {
   HashMap<Integer, Product> products = new HashMap<>();
   void addProduct(Product p) {
      products.put(p.productId, p);
   }
   void updateProduct(int id, int qty) {
      products.get(id).quantity = qty;
   }
   void deleteProduct(int id) {
      products.remove(id);
```

```
    }
    void display() {
        for(Product p : products.values())
            System.out.println(p);
    }
    public static void main(String args[]) {
        InventoryManagement i = new InventoryManagement();
        i.addProduct(new Product(1,"Laptop",5,50000));
        i.addProduct(new Product(2,"Mobile",10,20000));
        i.updateProduct(1,8);
        i.deleteProduct(2);
        i.display();
    }
}
```

Exercise 2:

```
class Product {
    int id;
    String name;
    String category;
    Product(int id,String name,String category){
        this.id=id;
        this.name=name;
        this.category=category;
    }
}
public class SearchExample {
```

```
static int linearSearch(Product arr[], String key){
   for(int i=0;i<arr.length;i++){
      if(arr[i].name.equals(key))
         return i;
   }
   return -1;
}
static int binarySearch(Product arr[], String key){
   int low=0, high=arr.length-1;
   while(low<=high){
      int mid=(low+high)/2;
      if(arr[mid].name.equals(key))
         return mid;
      if(arr[mid].name.compareTo(key)<0)
         low=mid+1;
      else
         high=mid-1;
   }
   return -1;
}
public static void main(String args[]){
   Product p[]={
      new Product(1,"Apple","Mobile"),
      new Product(2,"Laptop","Computer"),
      new Product(3,"Watch","Accessory")
   };
   System.out.println(linearSearch(p,"Laptop"));
```

```
        System.out.println(binarySearch(p,"Watch"));
    }
}
```

Exercise3:

```
class Order {
    int id;
    String name;
    double price;
    Order(int id,String name,double price){
        this.id=id;
        this.name=name;
        this.price=price;
    }
}
public class SortingOrders {
static void bubbleSort(Order arr[]){
    for(int i=0;i<arr.length-1;i++){
        for(int j=0;j<arr.length-i-1;j++){
            if(arr[j].price>arr[j+1].price){
                Order temp=arr[j];
                arr[j]=arr[j+1];
                arr[j+1]=temp;
            }
        }
    }
}
```

```
static void quickSort(Order arr[],int low,int high){
   if(low<high){
      double pivot=arr[high].price;
      int i=low-1;
      for(int j=low;j<high;j++){
         if(arr[j].price<pivot){
            i++;
            Order temp=arr[i];
            arr[i]=arr[j];
            arr[j]=temp;
         }
      }
      Order temp=arr[i+1];
      arr[i+1]=arr[high];
      arr[high]=temp;
      int pi=i+1;
      quickSort(arr,low,pi-1);
      quickSort(arr,pi+1,high);
   }
}
public static void main(String args[]){
Order orders[]={
new Order(1,"A",500),
new Order(2,"B",1000),
new Order(3,"C",200)
};
bubbleSort(orders);
```

```
for(Order o:orders)
System.out.println(o.price);
}
}
```

Exercise 4:

```
class Employee{
int id;
String name;
String position;
double salary;
Employee(int id,String name,String pos,double sal){
this.id=id;
this.name=name;
this.position=pos;
this.salary=sal;
}
}
public class EmployeeManagement{
Employee emp[]=new Employee[5];
int count=0;
void add(Employee e){
emp[count++]=e;
}
void search(int id){
for(int i=0;i<count;i++){
if(emp[i].id==id)
System.out.println(emp[i].name);
```

```
}
}
void display(){
for(int i=0;i<count;i++)
System.out.println(emp[i].name);
}
void delete(int id){
for(int i=0;i<count;i++){
if(emp[i].id==id){
emp[i]=emp[count-1];
count--;
}
}
public static void main(String args[]){
EmployeeManagement e=new EmployeeManagement();
e.add(new Employee(1,"Ram","Developer",50000));
e.add(new Employee(2,"Sam","Tester",40000));
e.display();
e.search(1);
e.delete(2);
}
}
```

Exercise 5:

```
class Task {
   int id;
   String name;
   String status;
```

```
  Task(int id,String name,String status){
    this.id=id;
    this.name=name;
    this.status=status;
  }
}
class Node {
  Task task;
  Node next;
  Node(Task task){
    this.task=task;
  }
}
public class TaskManagement {
  Node head;
  void add(Task t){
    Node newNode=new Node(t){
    newNode.next=head;
    head=newNode;
  }
  void search(int id){
    Node temp=head;
    while(temp!=null){
      if(temp.task.id==id)
        System.out.println(temp.task.name);
      temp=temp.next;
    }
  }
```

```
void display(){
   Node temp=head;
   while(temp!=null){
      System.out.println(
      temp.task.id+" "+temp.task.name+" "+temp.task.status);
      temp=temp.next;
   }
}
void delete(int id){
   Node temp=head;
   Node prev=null;
   while(temp!=null){
      if(temp.task.id==id){
         if(prev==null)
            head=temp.next;
         else
            prev.next=temp.next;
         break;
      }
      prev=temp;
      temp=temp.next;
   }
}
public static void main(String args[]){
   TaskManagement t=new TaskManagement();
   t.add(new Task(1,"Coding","Pending"));
   t.add(new Task(2,"Testing","Done"));
   t.display();
```

```
        t.search(1);
        t.delete(2);
        t.display();
    }
}
```

Exercise 6:

```
class Book {
    int id;
    String title;
    String author;
    Book(int id,String title,String author){
        this.id=id;
        this.title=title;
        this.author=author;
    }
}
public class LibrarySearch {
static void linearSearch(Book books[],String key){
    for(Book b:books){
        if(b.title.equals(key))
            System.out.println("Found "+b.title);
    }
}
static void binarySearch(Book books[],String key){
    int low=0;
    int high=books.length-1;
    while(low<=high){
```

```
        int mid=(low+high)/2;
        if(books[mid].title.equals(key)){
            System.out.println("Found "+books[mid].title);
            return;
        }
        if(books[mid].title.compareTo(key)<0)
            low=mid+1;
        else
            high=mid-1;
    }
}
public static void main(String args[]){
Book books[]={
new Book(1,"Apple","Author1"),
new Book(2,"Java","Author2"),
new Book(3,"Python","Author3")
};
linearSearch(books,"Java");
binarySearch(books,"Python");
}
}
```

Exercise 7:

```
public class FinancialForecast {
static double calculateFutureValue(
double amount,double rate,int years){
if(years==0)
```

```
return amount;
return calculateFutureValue(
amount + (amount * rate),
rate,
years-1);
}
public static void main(String args[]){
double amount=10000;
double rate=0.10;
int years=5;
double result=
calculateFutureValue(amount,rate,years);
System.out.println(
"Future Value: "+result);
}
}
```